## TÉCNICO SUPERIOR EN DESARROLLO DE APLICACIONES WEB

### DESARROLLO WEB EN ENTORNO CLIENTE

### Primera Evaluación · Examen Parcial 1 - Fecha: 22/10/2026

**Nombre y apellidos:**

**Código de entrega (te lo da el script al entregar):**

# 🔴 EXAMEN DE DWEC: MODELO A

## Parte B · Práctica

**65 minutos · 10 puntos · 7 funciones en `examen.js`** · Sin internet, sin IA y sin apuntes.

## 1 · Abrir el examen

**1.** En la web del módulo, **jmfiorisies.github.io/dwec** → UD 3 → «Examen Parcial 1 · Parte B», descarga **`crear-examen-A.sh`**.

**2.** Abre una terminal (**Ctrl+Alt+T**) y escribe:

```
cd ~/Descargas
bash crear-examen-A.sh
```

**3.** Se abren solas dos ventanas: **VS Code** con la carpeta `examen1` y la **página del examen** (`http://localhost:6001`). Si falta alguna o sale un ✖, llama al profesor.

**4. Desactiva internet** (wifi y cable) delante del profesor.

## 2 · Hacer el examen

- Escribe tu nombre en la línea `Alumno/a:` de `examen.js` y completa las 7 funciones. Cada enunciado está encima de su función (y en esta hoja). Son independientes: si te atascas, pasa a la siguiente.
- Guarda con **Ctrl+S**. Una función a medias también puntúa.
- Para probar, escribe en `pruebas.js` y mira la consola del navegador (**F12**). Ese fichero no se corrige.

## 3 · Cerrar y entregar

**1.** Guarda y escribe en la terminal:

```
bash ~/examen1/examen.sh entregar
```

**2.** Escribe tu nombre y primer apellido. Sale un **código de entrega**: **apúntalo en tu hoja**, junto a tu nombre.

**3.** Avisa al profesor. Cuando active internet, envía por correo a **jcorfio941@g.educaand.es** el fichero `examen1-A-….tar.gz` que está en el **Escritorio** (adjunto, asunto «Examen DWEC P1 · tu nombre»). No vuelvas a empaquetar.

## 🆘 Si algo falla

| Problema | Qué hacer |
|---|---|
| Has cerrado VS Code | `bash ~/examen1/examen.sh code` |
| Has cerrado la página | `bash ~/examen1/examen.sh web` |
| La página no carga | `bash ~/examen1/examen.sh arrancar` |
| La página dice «No se puede cargar tu código» | Error de sintaxis en `examen.js`: busca el subrayado rojo |
| Cualquier otro ✖ | Llama al profesor |

## ⛔ Normas

- Usa solo el VS Code del examen y solo la carpeta `~/examen1`. Nada de IA, apuntes ni código ajeno: **usar IA supone suspender**.
- Queda registrado en tu entrega: si cierras el VS Code del examen o abres otro, si pegas mucho código de golpe, si se activa internet, y una copia de `examen.js` cada minuto.
- No toques `datos.js`, `main.js`, `index.html`, `estilos.css`, `Dockerfile`, `docker-compose.yml`, `.vscode/` ni el registro (`registro.sh`, `.registro/`).

## 🌱 Contexto: vivero «Raíces»

Tienes un array `plantas` en `datos.js` (no se modifica). Cada planta tiene esta forma:

```
{ id: 1, nombre: "Monstera", tipo: "interior", precio: 24, stock: 5,
riegoSemanal: 1 }
```

- `tipo`: `"interior"`, `"exterior"` o `"suculenta"`.
- `precio`: en euros, para el tamaño `"mediana"`.
- `riegoSemanal`: puede valer `0` (no necesita riego) o **no existir** (no tenemos el dato).

## 🧩 Ejercicios (los mismos enunciados que en `examen.js`)

### Ejercicio 1 (1 punto) · filter + map

```
Devuelve un array con los NOMBRES de las plantas que tienen stock 0.
  nombresAgotados(plantas) → ["Potus", "Echeveria"]

nombresAgotados(lista)
```

### Ejercicio 2 (1 punto) · switch o ternario + parámetro por defecto

```
Devuelve el precio según el tamaño, redondeado a 2 decimales:
  "pequeña" → -20 %   ·   "mediana" → igual   ·   "grande" → +30 %
  Si no se indica tamaño, se entiende "mediana".
  Cualquier otro tamaño → null.
  precioPorTamano(10, "grande") → 13     precioPorTamano(10) → 10
Escríbela como FUNCIÓN DECLARADA (con la palabra `function`).

function precioPorTamano(precio, tamano)
```

### Ejercicio 3 (1,5 puntos) · find + ?. + ??

```
Busca la planta por id y devuelve un texto:
  "Monstera · riego: 1 veces/semana"
  "Aloe vera · riego: 0 veces/semana"      ← 0 es un dato válido
  "Lavanda · riego: sin datos"             ← la propiedad no existe
  "No existe la planta 99"                 ← id que no está en la lista

fichaRiego(lista, id)
```

### Ejercicio 4 (1,5 puntos) · reduce

```
Devuelve el valor del stock (precio × stock) SOLO de las plantas del tipo
indicado, redondeado a 2 decimales. Si no hay plantas de ese tipo, o la
lista está vacía, devuelve 0.
  valorPorTipo(plantas, "interior") → 174     (24×5 + 9.5×0 + 18×3)
  valorPorTipo(plantas, "cactus")   → 0
Escríbela como EXPRESIÓN DE FUNCIÓN: const valorPorTipo = function (...)
{...}

const valorPorTipo = function (lista, tipo)
```

### Ejercicio 5 (1,5 puntos) · inmutabilidad (map + spread)

```
Rebaja un `porcentaje` (por defecto 10) el precio de TODAS las plantas del
`tipo` indicado. Devuelve un ARRAY NUEVO igual que `lista`, con esos precios
rebajados y redondeados a 2 decimales. Las demás plantas quedan igual.
  rebajar(plantas, "exterior", 50) → Lavanda 3, Olivo 22.5, Romero 2.75
  rebajar(plantas, "interior")     → Monstera 21.6, Potus 8.55, Ficus 16.2
La lista original NO puede cambiar (ni el array ni sus objetos).

rebajar(lista, tipo, porcentaje)
```

### Ejercicio 6 (2 puntos) · closure + parámetro rest

```
crearRegistroRiego() devuelve un objeto con tres funciones que comparten un
registro de riegos PRIVADO (no accesible desde fuera):
  - regar(dia, ...ids): apunta que ese día se regaron las plantas con esos
    ids (cualquier cantidad) y devuelve cuántas plantas se regaron ese día.
  - vecesRegada(id): en cuántos riegos aparece esa planta (0 si en ninguno).
  - dias(): devuelve un ARRAY NUEVO con los días de cada riego, en orden.
Ejemplo:
  const r = crearRegistroRiego();
  r.regar("lunes", 1, 2, 8)   → 3
  r.regar("jueves", 1)        → 1
  r.vecesRegada(1)            → 2        r.vecesRegada(4) → 0
  r.dias()                    → ["lunes", "jueves"]

crearRegistroRiego()
```

### Ejercicio 7 (1,5 puntos) · funciones de alto orden

```
a) filtrarCon(lista, criterio): devuelve un array nuevo con los elementos
   para los que criterio(elemento) es true. PROHIBIDO usar .filter():
   hazlo con un bucle (for...of) y spread.
b) masBaratasQue(max): devuelve una FUNCIÓN que recibe una planta y dice
   si su precio es menor o igual que `max`.
Uso:
  filtrarCon(plantas, masBaratasQue(6)).map((p) => p.nombre)
  → ["Lavanda", "Echeveria", "Romero"]

filtrarCon(lista, criterio)
masBaratasQue(max)
```